Name : Sahil Jakhariya
Roll No : 63
Class : D15C
Batch : C

MLDL Experiment 05

Aim : Implementation of Support Vector Machine (SVM) for Classification with Hyperparameter Tuning

1.Dataset Source

Dataset Name: **Bank Marketing Dataset (Deposit Prediction Version)**

Source:
Kaggle Version Used: Bank

2.Dataset Description

The Bank Marketing dataset contains data related to direct marketing campaigns of a Portuguese banking institution.

Dataset Size

- Total Samples: 11,162
- Total Features: 16 input features
- Target Variable: deposit

Target Variable

- 1 → Customer subscribed to term deposit
- 0 → Customer did not subscribe

Important Features

- age – Age of client
- job – Type of job
- marital – Marital status
- education – Education level

- balance – Average yearly balance
- housing – Housing loan
- loan – Personal loan
- duration – Last contact duration
- campaign – Number of contacts
- poutcome – Outcome of previous campaign

The dataset contains both categorical and numerical attributes. One-hot encoding was applied to categorical variables before training the model.

3.Mathematical Formulation of SVM

Support Vector Machine aims to find the optimal separating hyperplane:

$$w^T x + b = 0$$

Objective Function (Soft Margin SVM)

$$\min \frac{1}{2}||w||^2 + C\sum \xi_i$$

Where:

- (w) = Weight vector
- (b) = Bias
- (C) = Regularization parameter
- (\xi_i) = Slack variables

Kernel Trick

For non-linear separation, kernel functions are used.

RBF Kernel:

$$K(x, x') = exp(-\gamma||x - x'||^2)$$

Where:

- (\gamma) controls influence of individual training samples.

4.Algorithm Limitations

- Computationally expensive for large datasets
- Sensitive to hyperparameters (C, gamma)
- Requires proper feature scaling
- Difficult to interpret compared to linear models
- Performance affected by class imbalance

5. Methodology / Workflow

Step 1: Data Loading

Loaded dataset using pandas.

Step 2: Data Preprocessing

- Converted target variable (`deposit`) to binary (1/0)
- Applied One-Hot Encoding to categorical variables
- Performed Train-Test Split (80:20)
- Applied StandardScaler for feature scaling

Step 3: Model Training

Implemented SVM classifier using `SVC`.

Step 4: Hyperparameter Tuning

Used GridSearchCV to tune:

- C = [0.1, 1, 10]
- Kernel = ['linear', 'rbf']
- Gamma = ['scale', 0.01]

Best parameters were selected based on cross-validation performance.

Step 5: Model Evaluation

Evaluated using:

- Accuracy
- Classification Report
- Confusion Matrix
- ROC Curve (AUC)
- Precision-Recall Curve (Average Precision)

6. Performance Analysis

The following evaluation metrics were used:

1. Confusion Matrix

Shows:

- True Positives
- True Negatives ● False Positives
- False Negatives

It provides insight into classification errors.

2.Accuracy

$$Accuracy = \frac{TP + TN}{Total}$$

Measures overall correctness of the model.

3.ROC Curve and AUC

- ROC plots True Positive Rate vs False Positive Rate.
- AUC measures classifier's ability to distinguish classes.

Interpretation:

- 0.5 → Random
- 0.7 → Acceptable
- 0.8+ → Good
- 0.9+ → Excellent

4.Precision-Recall Curve

- Useful for imbalanced datasets.
- Average Precision (AP) summarizes the precision-recall tradeoff.

This is especially important since the dataset has class imbalance.

7. Hyperparameter Tuning

Hyperparameters tuned:

C (Regularization Parameter)

- Small C → More regularization, wider margin
- Large C → Less regularization, tighter margin

Kernel

- Linear → Straight decision boundary
- RBF → Non-linear decision boundary

Gamma

- Small gamma → Smooth decision boundary
- Large gamma → Complex decision boundary

GridSearchCV selected optimal parameters, which improved performance compared to default SVM.

8. Impact of Hyperparameter Tuning

- Improved classification accuracy
- Optimized margin separation
- Better class distinction
- Improved ROC-AUC score

Thus, hyperparameter tuning significantly enhanced model performance.

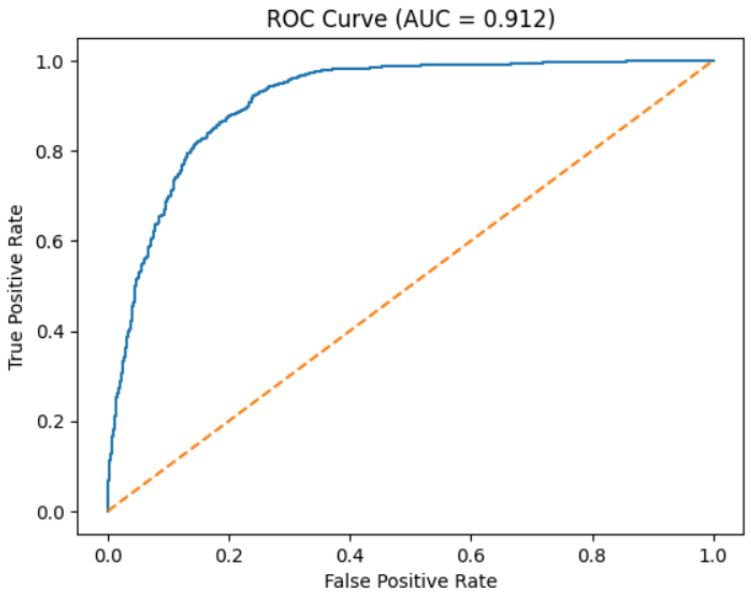
Code

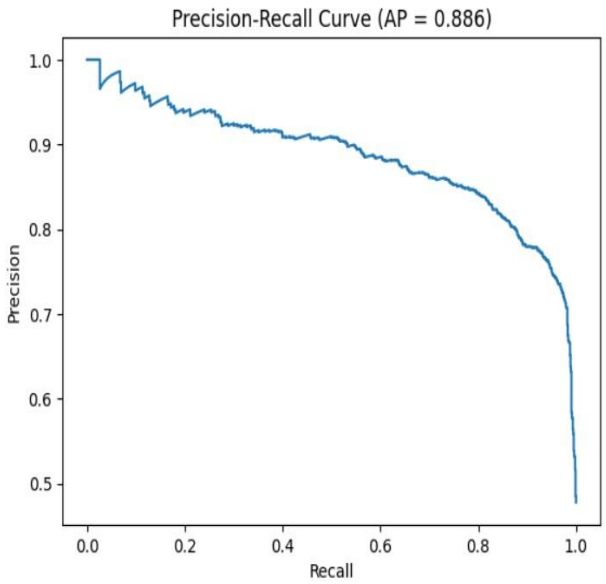
```
import pandas as pd import
numpy as np import seaborn
as sns import
matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler from
sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# ----------------------------
# 1. Load Dataset (Correct Way)
# ----------------------------
df = pd.read_csv("bank.csv")   # removed sep=";"
print("Dataset Shape:", df.shape) print(df.head())

# ----------------------------
# 2. Preprocessing
# ----------------------------

# Convert target variable to binary df['deposit']
= df['deposit'].map({'yes':1, 'no':0})

# One-hot encoding for categorical variables
df = pd.get_dummies(df, drop_first=True)

# Separate features and target
X = df.drop("deposit", axis=1) y
= df["deposit"]
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Feature Scaling scaler
= StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# ----------------------------
# 3. SVM + Hyperparameter Tuning
# ----------------------------

param_grid = {
    'C': [0.1, 1, 10],
    'kernel': ['linear', 'rbf'],
    'gamma': ['scale', 0.01]
}

grid = GridSearchCV(SVC(), param_grid, cv=5, n_jobs=-1) grid.fit(X_train,
y_train)

print("\nBest Parameters Found:", grid.best_params_)

best_model = grid.best_estimator_

# ----------------------------
# 4. Evaluation
# ----------------------------

y_pred = best_model.predict(X_test)

print("\nAccuracy:", accuracy_score(y_test, y_pred)) print("\nClassification
Report:\n", classification_report(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5,4)) sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix") plt.xlabel("Predicted") plt.ylabel("Actual") plt.show()
from sklearn.metrics import roc_curve, roc_auc_score, precision_recall_curve,
average_precision_score

# Get decision scores y_scores =
best_model.decision_function(X_test)

# ROC Curve
fpr, tpr, _ = roc_curve(y_test, y_scores) roc_auc
= roc_auc_score(y_test, y_scores)
plt.figure() plt.plot(fpr, tpr)
plt.plot([0,1],[0,1],'--') plt.title(f"ROC Curve
(AUC = {roc_auc:.3f})") plt.xlabel("False
Positive Rate") plt.ylabel("True Positive
Rate") plt.show()
```

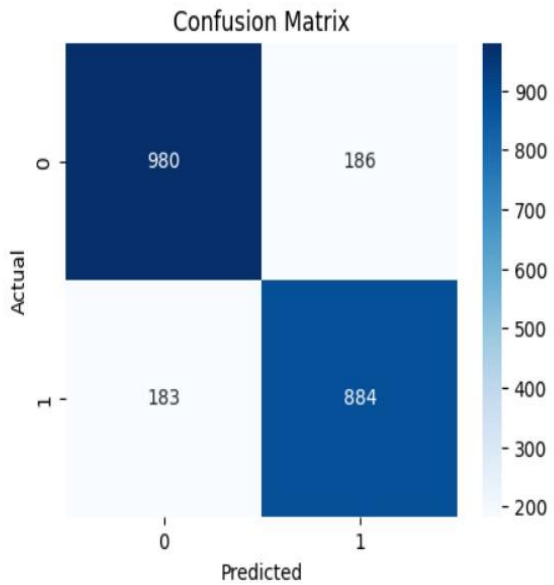
```
# Precision-Recall Curve precision, recall, _ =
precision_recall_curve(y_test, y_scores) ap =
average_precision_score(y_test, y_scores)
plt.figure() plt.plot(recall, precision)
plt.title(f"Precision-Recall Curve (AP = {ap:.3f})")
plt.xlabel("Recall") plt.ylabel("Precision")
plt.show()
```

OUTPUT






```
Classification Report:
              precision    recall  f1-score   support
...
           0       0.84      0.84      0.84      1166
           1       0.83      0.83      0.83      1067

    accuracy                           0.83      2233
   macro avg       0.83      0.83      0.83      2233
weighted avg       0.83      0.83      0.83      2233
```



Conclusion

Support Vector Machine (SVM) was successfully implemented for classification on the Bank Marketing dataset.

- Applied preprocessing and encoding
- Implemented SVM classifier
- Performed hyperparameter tuning using GridSearchCV
- Evaluated model using advanced metrics (ROC, AUC, Precision-Recall Curve)

The experiment demonstrates that SVM is an effective classification algorithm when properly tuned and evaluated.